Neural Compression for Power Spectral Density

Martín Ramírez Espinosa, Sergio Alejandro Gil Ríos

February 2026

1 Introduction

Wideband spectrum monitoring based on low-cost Software Defined Radios (SDRs) is a practical approach to assess occupancy in the FM broadcast band and to support regulatory and operational compliance tasks. In these systems, a sensing node continuously acquires complex baseband samples, computes spectral features (in particular, a Power Spectral Density (PSD) estimate), and reports results to a remote server for visualization and decision-making.

A key bottleneck is communication and storage: sending raw I/Q streams or high-resolution PSD frames at high cadence can easily exceed uplink capacity and backend storage/processing budgets. This work formulates PSD compression as a rate–distortion problem and proposes a VQ-VAE-based system model that learns a compact discrete representation (bitstream) of the PSD while preserving the quantities used for FM spectrum sensing, such as occupancy maps and emission-related features.

2 Problem Formulation

2.1 System model

We consider a sensing node that, at discrete time instants t, computes a PSD estimate of a received process $x(t)$ over a band of interest. Denote by

$$\mathbf{S}_t \triangleq [S_t(\omega_1), \ldots, S_t(\omega_N)]^\top \in \mathbb{R}_+^N \tag{1}$$

the discretized PSD sampled on a uniform frequency grid $\{\omega_n\}_{n=1}^N$ covering the monitored band. The node must transmit a compressed description of $\mathbf{S}_t$ to a cloud service under a bitrate budget, where the cloud reconstructs $\hat{\mathbf{S}}_t$ and performs spectrum-sensing analytics.

2.2 Sensing-driven quantities derived from the PSD

The remote monitor uses the reconstructed PSD to derive spectrum-sensing quantities relevant to FM monitoring. Typical examples include:

$$\omega_{\text{pk}} \triangleq \arg\max_{\omega} S_t(\omega) \qquad \text{(peak/candidate carrier)} \tag{2}$$

$$\omega_{\text{cent}} \triangleq \frac{\sum_{n=1}^N \omega_n S_t(\omega_n)}{\sum_{n=1}^N S_t(\omega_n)} \qquad \text{(spectral centroid).} \tag{3}$$

Additionally, an occupancy map can be obtained by comparing the PSD to a (possibly frequency-dependent) noise-floor estimate $N_b(\omega)$ plus a margin γ:

$$O_t(\omega_n) \triangleq \mathbf{1}\{S_t(\omega_n) > N_b(\omega_n) + \gamma\}. \tag{4}$$

Connected occupied regions define candidate emissions and may be used to estimate an occupied bandwidth B_{occ} (e.g., by thresholded support width or by a power-containment rule).

2.3 Bit budget and rate–distortion objective

Let $L_{b_s}(t)$ and $L_{b_t}(t)$ denote the number of bits used by the main bitstream and the side information, respectively. The total payload per PSD frame is

$$L_b(t) = L_{b_s}(t) + L_{b_t}(t). \tag{5}$$

We seek to compress the PSD while preserving both waveform-level fidelity and sensing-level decisions derived from the PSD.

A rate–distortion formulation is:

$$\min_{\theta,\Phi,\mathcal{E}} \ \mathbb{E}[\mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t)] \quad \text{s.t.} \quad \mathbb{E}[L_b(t)] \le R_{\max}, \tag{6}$$

or equivalently the Lagrangian form:

$$\min_{\theta,\Phi,\mathcal{E}} \ \mathbb{E}\Big[\lambda\, \mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t) + L_b(t)\Big], \tag{7}$$

where $\lambda > 0$ controls the rate–distortion trade-off.

2.4 Distortion tailored to spectrum sensing

Because the cloud ultimately performs sensing on $\hat{\mathbf{S}}_t$, we define $\mathcal{D}$ to penalize errors that matter for monitoring:

1. **PSD fidelity.** A normalized reconstruction error (often in log-PSD) such as

$$\mathcal{D}_{\text{psd}} = \frac{1}{N} \sum_{n=1}^{N} \big(\psi(S_t(\omega_n)) - \psi(\hat{S}_t(\omega_n))\big)^2, \tag{8}$$

   with $\psi(\cdot)$ an optional dynamic-range map (e.g., $\log(\epsilon + \cdot)$).

2. **Occupancy consistency.** Let $O_t(\omega_n)$ and $\hat{O}_t(\omega_n)$ be computed from S_t and $\hat{S}_t$ using the same noise-floor rule. Penalize mismatches, e.g., by a weighted binary cross-entropy (or FP/FN-weighted) loss across frequency bins.

3. **Feature preservation.** Penalize deviations in sensing features such as peak and centroid:

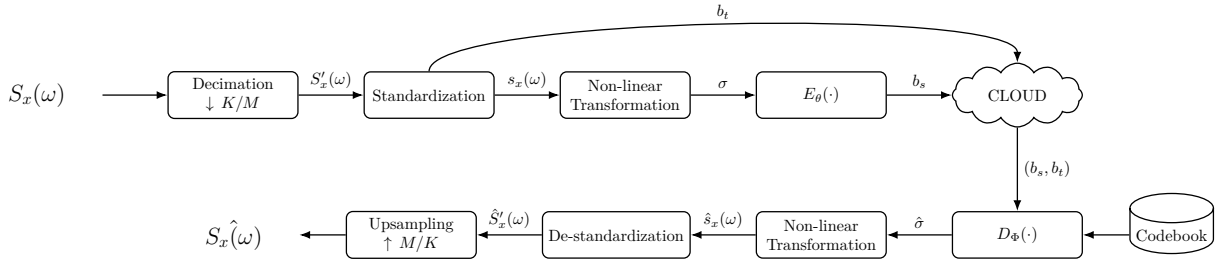
$$\mathcal{D}_{\text{feat}} = \alpha_{\text{pk}} |\omega_{\text{pk}} - \hat{\omega}_{\text{pk}}| + \alpha_{\text{cent}} |\omega_{\text{cent}} - \hat{\omega}_{\text{cent}}| + \alpha_B |B_{\text{occ}} - \hat{B}_{\text{occ}}|. \tag{9}$$

A practical overall distortion is a weighted sum,

$$\mathcal{D} = \beta_{\text{psd}} \mathcal{D}_{\text{psd}} + \beta_{\text{occ}} \mathcal{D}_{\text{occ}} + \beta_{\text{feat}} \mathcal{D}_{\text{feat}}. \tag{10}$$

3 System model proposals

3.1 Learned compression with a VQ-VAE



3.1.1 Compression pipeline

For each PSD frame $\mathbf{S}_t$, the sensing node applies a deterministic pre-/post-processing chain and a learned VQ-VAE bottleneck.

Decimation (frequency downsampling). A deterministic operator reduces frequency resolution:

$$\mathbf{S}'_t = \mathcal{D}_{K/M}(\mathbf{S}_t) \in \mathbb{R}_+^{N'}, \qquad N' \approx N \cdot \frac{K}{M}. \tag{11}$$

Standardization and side information. We standardize the decimated PSD using parameters $\mathbf{t}_t$ (e.g., mean/scale, min/max, or blockwise gains),

$$\mathbf{s}_t = \mathrm{Std}(\mathbf{S}'_t; \mathbf{t}_t), \tag{12}$$

and transmit a quantized representation of these parameters as side-information bits b_t so the cloud can invert the operation.

Non-linear transformation. To better match the dynamic range of PSD values to the quantizer and the neural model, we apply an invertible pointwise mapping

$$\boldsymbol{\sigma}_t = g_{\mathrm{nl}}(\mathbf{s}_t), \qquad \mathbf{s}_t = g_{\mathrm{nl}}^{-1}(\boldsymbol{\sigma}_t). \tag{13}$$

Examples include log-like compressions and other monotone mappings that reduce heavy tails.

VQ-VAE encoder, vector quantization, and bitstream b_s. The encoder produces latent features $\mathbf{z}_t = E_\theta(\boldsymbol{\sigma}_t)$ which are partitioned into blocks. Each block is quantized to the nearest codeword in a learned codebook $\mathcal{E} = \{\mathbf{e}_1, \ldots, \mathbf{e}_{|\mathcal{E}|}\}$:

$$\mathbf{k}_t = Q_{\mathcal{E}}(\mathbf{z}_t), \qquad \hat{\mathbf{z}}_t = \mathcal{E}[\mathbf{k}_t]. \tag{14}$$

The indices $\mathbf{k}_t$ are entropy-coded into a primary bitstream b_s (main payload). The total transmitted/stored payload per PSD frame is therefore (b_s, b_t).

Decoder and inverse processing. At the cloud, the decoder reconstructs

$$\hat{\boldsymbol{\sigma}}_t = D_\Phi(\hat{\mathbf{z}}_t), \tag{15}$$

$$\hat{\mathbf{s}}_t = g_{\mathrm{nl}}^{-1}(\hat{\boldsymbol{\sigma}}_t), \tag{16}$$

$$\hat{\mathbf{S}}'_t = \mathrm{DeStd}(\hat{\mathbf{s}}_t; \mathbf{t}_t), \tag{17}$$

$$\hat{\mathbf{S}}_t = \mathcal{U}_{M/K}(\hat{\mathbf{S}}'_t), \tag{18}$$

where $\mathcal{U}_{M/K}$ is an upsampling/interpolation operator restoring the original frequency grid.

3.1.2 VQ-VAE training losses (codebook and commitment)

In addition to the rate–distortion objective above, VQ-VAE training typically includes terms that stabilize the discrete bottleneck. Let $\mathrm{sg}[\cdot]$ denote the stop-gradient operator. A common choice is:

$$\mathcal{L}_{\mathrm{VQ}} = \|\mathrm{sg}[\mathbf{z}_t] - \hat{\mathbf{z}}_t\|_2^2 + \beta_{\mathrm{com}}\|\mathbf{z}_t - \mathrm{sg}[\hat{\mathbf{z}}_t]\|_2^2, \tag{19}$$

where the first term updates the codebook and the second (commitment) term encourages encoder outputs to stay close to selected codewords.

Overall training problem. Combining the sensing-aware distortion, the bit budget, and the VQ terms yields the final learning problem:

$$\min_{\theta,\Phi,\mathcal{E}} \ \mathbb{E}\Big[\lambda\, \mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t) + L_b(t)\Big] + \eta\, \mathbb{E}\big[\mathcal{L}_{\mathrm{VQ}}\big], \tag{20}$$

with hyperparameters $\lambda, \eta, \beta_{\mathrm{com}}$ and the sensing-aware weights inside $\mathcal{D}$.

3.2 Deterministic compression with FWHT sparsification

As an alternative to learned quantization, we consider a deterministic transform coder based on the *Fast Walsh–Hadamard Transform (FWHT)*. The method applies the same front-end preprocessing used above (decimation, standardization, and the optional non-linear mapping) to obtain $\boldsymbol{\sigma}_t \in \mathbb{R}^{N'}$ and transmits the corresponding side-information bits b_t. The main bitstream b_s is then formed by sparsifying the FWHT coefficients of $\boldsymbol{\sigma}_t$.

Hadamard transform. Let N_H be a power of two. If N' is not a power of two, we zero-pad $\boldsymbol{\sigma}_t$ to length $N_H = 2^{\lceil \log_2 N' \rceil}$ (and discard padded bins after inversion). Let $H_{N_H} \in \{\pm 1\}^{N_H \times N_H}$ denote the Hadamard matrix and use the orthonormal transform $\frac{1}{\sqrt{N_H}} H_{N_H}$. The sequency-domain coefficients are

$$\mathbf{c}_t = \frac{1}{\sqrt{N_H}}\, H_{N_H}\, \boldsymbol{\sigma}_t. \tag{21}$$

Top-K sparsification and bitstream b_s. Define Ω_t as the index set of the K largest-magnitude coefficients of $\mathbf{c}_t$. We form a sparse vector $\tilde{\mathbf{c}}_t$ such that $(\tilde{\mathbf{c}}_t)_i = (\mathbf{c}_t)_i$ for $i \in \Omega_t$ and zero otherwise. The transmitted payload b_s contains (i) the locations Ω_t (e.g., as an index list or a bitmask) and (ii) quantized coefficient values $\mathcal{Q}((\mathbf{c}_t)_i)$ for $i \in \Omega_t$. In practice, Ω_t can be computed efficiently without a full sort (e.g., via selection algorithms such as Quickselect).

Reconstruction. The cloud reconstructs the transformed PSD by the inverse FWHT:

$$\hat{\boldsymbol{\sigma}}_t = \frac{1}{\sqrt{N_H}} H_{N_H}\, \tilde{\mathbf{c}}_t, \tag{22}$$

and then applies the same inverse processing as before using b_t: $\hat{\mathbf{s}}_t = g_{\mathrm{nl}}^{-1}(\hat{\boldsymbol{\sigma}}_t)$, $\hat{\mathbf{S}}'_t = \mathrm{DeStd}(\hat{\mathbf{s}}_t; \mathbf{t}_t)$, and $\hat{\mathbf{S}}_t = \mathcal{U}_{M/K}(\hat{\mathbf{S}}'_t)$.

Complexity and operating point. FWHT has $\mathcal{O}(N_H \log N_H)$ time complexity and uses only additions/subtractions. The rate–distortion trade-off is controlled mainly by K and the coefficient quantizer $\mathcal{Q}(\cdot)$, with no training required.

3.3 KL/PCA compression over frequency grid

As a third model, we include a *Karhunen–Loève (KL) / PCA* frequency-domain representation implemented as a module-based codec in the unified library. The method uses a covariance projection over the frequency grid, projects PSD snapshots to a low-rank subspace, and reconstructs from retained coefficients in that subspace.

KL basis on weighted frequency grid. Given frequencies $\{f_i\}_{i=1}^{M}$ and quadrature weights $\{w_i\}_{i=1}^{M}$, define the weighted kernel matrix

$$K_w(i,j) = k(f_i, f_j)\sqrt{w_i w_j}, \tag{23}$$

with $k(\cdot,\cdot)$ chosen as Matérn-3/2 (single or composite scale). Let $K_w \mathbf{v}_n = \lambda_n \mathbf{v}_n$, sorted by descending λ_n. Retaining R dominant modes, the KL feature matrix is

$$\Phi_{\mathrm{KL}}(i,n) = \sqrt{\lambda_n}\, \phi_n(f_i), \quad \phi_n(f_i) = \frac{v_n(i)}{\sqrt{w_i}}. \tag{24}$$

Low-rank PSD model and posterior reconstruction. For each frame, the excess PSD is modeled as

$$\mathbf{s}_t \approx \Phi_{\mathrm{KL}} \boldsymbol{\xi}_t, \qquad \boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, I_R), \tag{25}$$

with observation model

$$\mathbf{y}_t = A\boldsymbol{\xi}_t + \boldsymbol{\varepsilon}_t, \qquad A \equiv \Phi_{\mathrm{KL}}, \quad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \sigma_\varepsilon^2 I). \tag{26}$$

The posterior in coefficient space is Gaussian:

$$\Sigma_{\xi|y,t} = \left(I_R + \frac{A^\top A}{\sigma_\varepsilon^2} \right)^{-1}, \tag{27}$$

$$\mu_{\xi|y,t} = \Sigma_{\xi|y,t} \frac{A^\top y_t}{\sigma_\varepsilon^2}, \tag{28}$$

and reconstructed PSD mean is

$$\hat{\mathbf{s}}_t = A\, \mu_{\xi|y,t}. \tag{29}$$

This provides a deterministic dimensionality reduction path (KL/PCA basis) plus uncertainty-aware inference in the reduced space.

Implementation mapping (official three-model scope)

Model	Primary implementation path in repository
KL/PCA over frequency grid	`psd_compression/kl_pca/model.py` and `psd_compression/kl_pca/tasks.py`
Learned VAE compression	`VAE_implementation/scripts/01_preprocess.py` through `VAE_implementation/scripts/06_codec_benchmark.py` (plus UDP scripts)
Deterministic FWHT sparsification	`psd_compression/fwht/codec.py` and `psd_compression/fwht/tasks.py` (reference notebook kept at `experiments/fwht.ipynb`)